

Software Quality Assurance and Management

第1周：软件质量保证与管理导论

Week 1: Introduction to Software Quality Assurance & Management

By: Fahad Sabah

计算机学院,

北京工业大学,中国

College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



Contents

1. Introduction to the Course

2. Foundations: Quality Concepts and the Cost of Quality

3. Quality in Traditional Lifecycle Models

4. Quality in Agile & DevOps Lifecycles

5. Case Analysis: Quality Failures & Assignment Kick-off

目录

1. 课程介绍

2. 基础：质量概念与质量的成本

3. 传统生命周期模型中的质量

4. 敏捷与DevOps生命周期中的质量

5. 案例分析：质量失败与作业启动

第一周学习目标

Learning Objectives for Week 1

- ❑ **Defining software quality:** Define software quality, quality assurance (QA), quality control (QC), and quality management (QM), and distinguish between their roles.
- ❑ **QA vs QC vs QM:** Explain the key quality attributes and the cost of quality model.
- ❑ **Cost of Quality model:** Compare how quality practices are embedded in traditional, Agile, and DevOps lifecycles.
- ❑ **Interactive exercise & discussion:** Analyze a real-world software failure through the lens of quality management principles.
- ❑ **定义软件质量:** 定义软件质量, 质量保证 (QA), 质量控制 (QC) 和质量管理 (QM), 并区分它们的角色。
- ❑ **QA vs QC vs QM:** 解释关键的质量属性和质量成本模型。
- ❑ **质量成本模型:** 比较质量实践如何嵌入传统, 敏捷和DevOps生命周期。
- ❑ **互动练习与讨论:** 通过质量管理原则的视角分析现实世界的软件故障。

欢迎与会议概述

Welcome & Session Overview

Course Introduction:

Course title: Software Quality Assurance and Management

Course Credit(s): 32

Course Contents

Course Structure

four sessions every week (45 minutes each)

Introduction to the Instructor

课程介绍:

课程名称: 软件质量保证与管理

课程学分: 32

课程内容

课程结构

每周四次 (每次45分钟)

导师介绍

Week 1: Introduction to Software Quality Assurance & Management 第1周: 软件质量保证与管理导论

北京工业大学 软件质量保证与管理

Beijing University of Technology, Software Quality Assurance and Management

欢迎与会议概述

Welcome & Session Overview

Course Introduction:

Course title: Software Quality Assurance and Management



What is **Software Quality Assurance**?

Quality is an attribute of software that implies the software meets its specification

This definition is too simple for ensuring quality in software systems

- ☐ Software specifications are often incomplete or ambiguous
- ☐ Some quality attributes are difficult to specify
- ☐ Tension exists between some quality attributes, e.g. efficiency vs. reliability

What is **Software Project Management**?

Project management is the application of knowledge, skills, tools, and techniques to project activities to meet project requirements.

欢迎与会议概述

Welcome & Session Overview

教学内容及进度安排/Course Content & Schedule	
教学周 Week	教学内容 Contents
1	软件质量保证与管理导论 • 质量属性、QA vs QC vs QM、质量成本 • 生命周期中的质量视图（传统 vs 敏捷/DevOps） • 案例：healthcare.gov 等系统的质量失败分析 Introduction to Software Quality Assurance & Management • Quality attributes, QA vs QC vs QM, cost of quality • Quality views across lifecycles (traditional vs Agile/DevOps) • Case analysis: quality failures (e.g., healthcare.gov)
2	质量规划与基于风险的测试策略 • 质量计划编制、验收标准、测试策略制定 • 风险识别、评估与基于风险的测试规划 • 测试范围界定与工作分解（Test WBS） Quality Planning & Risk-Based Test Strategy • Quality planning, acceptance criteria, test strategy • Risk identification, assessment, and risk-based test planning • Test scope definition and Test WBS

Week 1: Introduction to Software Quality Assurance & Management 第1周：软件质量保证与管理导论

北京工业大学 软件质量保证与管理

Beijing University of Technology, Software Quality Assurance and Management

欢迎与会议概述

Welcome & Session Overview

3	验证与确认技术 • 静态测试：审查、走查、静态分析 • 动态测试技术：等价类划分、边界值分析、状态迁移测试 • 白盒测试与探索性测试简介 Verification & Validation Techniques • Static testing: reviews, walkthroughs, static analysis • Dynamic techniques: equivalence partitioning, boundary value, state transition • White-box testing and exploratory testing introduction
4	测试管理与敏捷测试实践 • 测试生命周期、测试进度与估算（测试点估算） • 敏捷测试四象限、Scrum/Kanban 中的测试角色 • 测试管理工具与缺陷生命周期（JIRA/Xray 示例） Test Management & Agile Testing Practices • Test lifecycle, test scheduling and estimation (test point estimation) • Agile Testing Quadrants, tester roles in Scrum/Kanban • Test management tools and defect lifecycle (JIRA/Xray examples)

欢迎与会议概述

Welcome & Session Overview

5	测试自动化与持续质量工程 • 测试自动化策略、框架选型 (Selenium 等) • CI/CD 流水线中的质量门与持续测试 • 容器化测试环境与基础设施即代码的验证 Test Automation & Continuous Quality Engineering • Test automation strategy, framework selection (Selenium, etc.) • Quality gates in CI/CD pipelines and continuous testing • Containerized test environments and Infrastructure-as-Code validation
6	质量度量、分析与改进 • 核心质量度量：缺陷密度、测试覆盖率、逃逸缺陷、MTTR • 质量分析技术与根本原因分析 (RCA) • 构建质量度量仪表盘 (Power BI/Grafana) Quality Metrics, Analysis & Improvement • Core metrics: defect density, test coverage, escaped defects, MTTR • Quality analysis techniques and root cause analysis (RCA) • Building quality dashboards (Power BI/Grafana)

欢迎与会议概述

Welcome & Session Overview

7	过程改进、标准与质量文化 • 过程改进模型与标准: CMMI、ISO 25010/9001、TMMi • 技术债务管理与质量技术债量化 • 质量文化、伦理责任 (AI公平性、数据隐私) 与合规 Process Improvement, Standards & Quality Culture • Improvement models and standards: CMMI, ISO 25010/9001, TMMi • Technical debt management and quality debt quantification • Quality culture, ethical responsibility (AI fairness, data privacy), and compliance
8	质量保证与管理毕业项目 • 制定完整的质量保证计划 (含风险分析、测试策略、度量与改进) • 在给定系统中实施质量门与关键度量 • 成果汇报: 20分钟小组演示 (含问答环节) Capstone Project in Software Quality Assurance & Management • Develop a complete quality assurance plan (risk, test strategy, metrics, improvement) • Implement quality gates and key metrics on a given system • Presentation: 20-minute group presentation with Q&A

欢迎与会议概述

Welcome & Session Overview

Course Structure

- ❑ Four sessions every week (45 minutes each)
- ❑ Three sessions for discussion on current topic
- ❑ One session for Project/Papers/Tools Discussion

课程结构

- ❑ 每周四次（每次45分钟）
- ❑ 三场讨论当前话题
- ❑ 一个项目/论文/工具讨论环节

欢迎与会议概述

Welcome & Session Overview

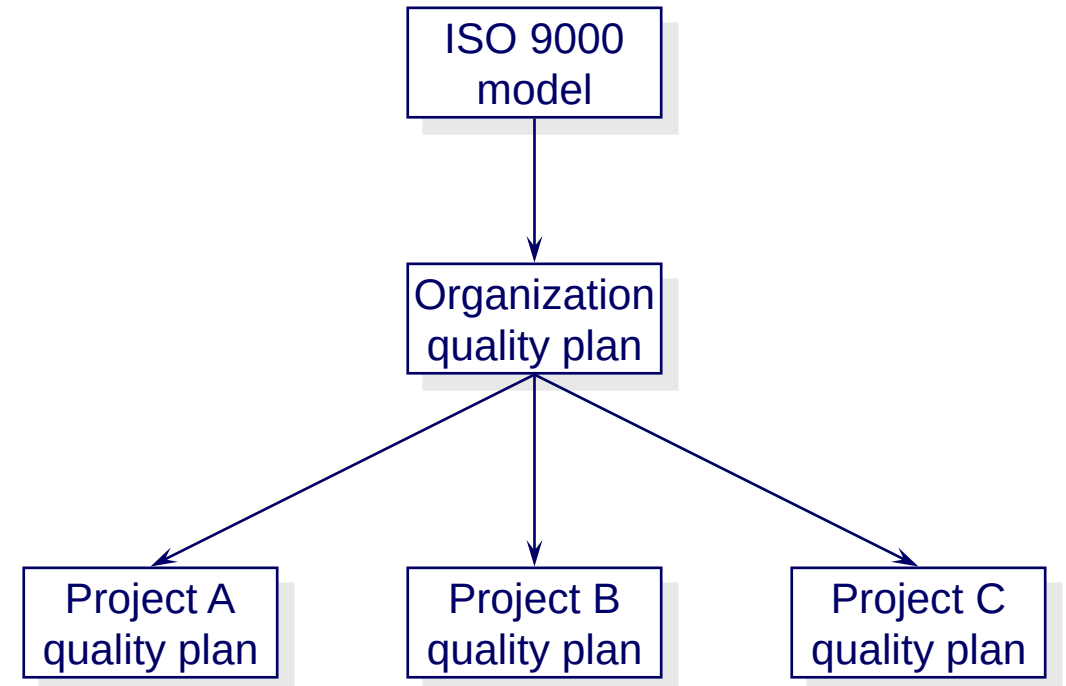
Quizzes	10%	Short, post-module online quizzes assessing key concepts (quality attributes, test techniques, CI/CD, metrics, standards, ethics).
Assignments (6 hands-on deliverables)	30%	❑ Week 2: Risk-Based Test Plan & Test WBS ❑ Week 3: Black-Box & White-Box Test Case Design ❑ Week 4: Agile Test Management & Defect Tracking (JIRA) ❑ Week 5: CI/CD Pipeline with Automated Quality Gates (Jenkins) ❑ Week 6: Quality Metrics Dashboard & Analysis Report (Power BI/Grafana) ❑ Week 7: Software Quality Maturity Assessment & Improvement Roadmap Evaluated on technical correctness, completeness, tool proficiency, and analytical depth.
Final Term Exam	30%	Open Book/Notes Exam, Evaluates theoretical mastery and ability to connect concepts.
Capstone Project	30%	Integrated quality assurance plan for a given system (risk analysis, test strategy, quality gates, metrics, improvement roadmap). 10-minute team presentation with live Q&A. Assessed on strategic coherence, technical rigor, quality of deliverables, communication, and teamwork.

欢迎与会议概述

Software Quality Attributes

- ☐ Safety
- ☐ Security
- ☐ Reliability
- ☐ Resilience
- ☐ Robustness
- ☐ Understandability
- ☐ Testability
- ☐ Adaptability

- ☐ Modularity
- ☐ Complexity
- ☐ Portability
- ☐ Usability
- ☐ Reusability
- ☐ Efficiency
- ☐ Learnability



欢迎与会议概述

SQA Activities

Applying technical methods

- ☐ To help the analyst achieve a high quality specification and a high quality design

Conducting formal technical reviews

- ☐ A stylized meeting conducted by technical staff with the sole purpose of uncovering quality problems

Testing Software

- ☐ A series of test case design methods that help ensure effective error detection

Enforcing standards

Controlling change

- ☐ Applied during software development and maintenance

Measurement

- ☐ Track software quality and assess the ability of methodological and procedural changes to improve software quality

Record keeping and reporting

- ☐ Provide procedures for the collection and dissemination of SQA information

SQA的优势

Advantages of SQA

- ❑ Software will have fewer latent defects, resulting in reduced effort and time spent during testing and maintenance
- ❑ Higher reliability will result in greater customer satisfaction
- ❑ Maintenance costs can be reduced
- ❑ Overall life cycle cost of software is reduced
- ❑ 软件的潜在缺陷会更少，从而减少测试和维护所需的精力和时间
- ❑ 更高的可靠性将带来更高的客户满意度
- ❑ 维护成本可以降低
- ❑ 软件的整体生命周期成本降低

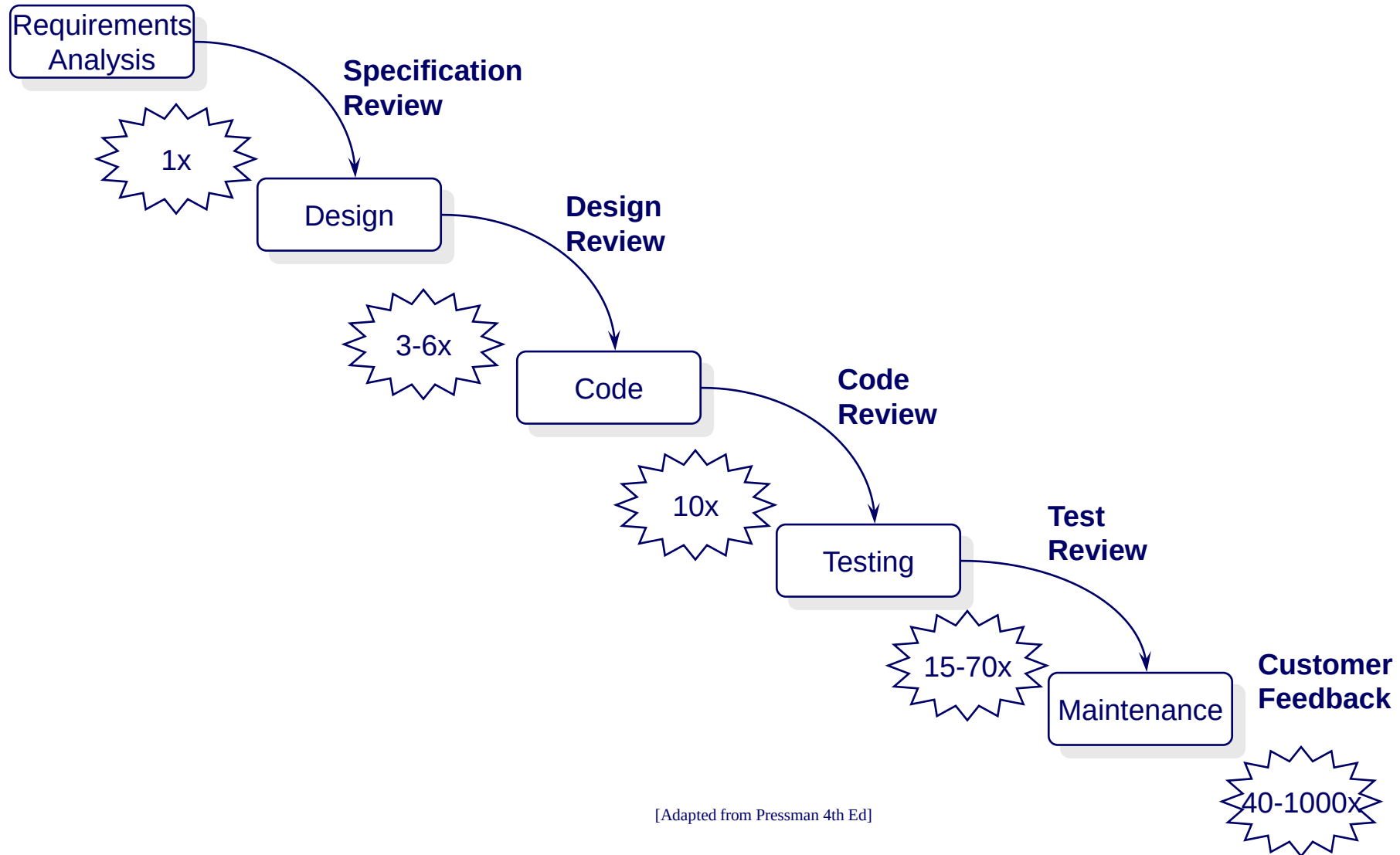
SQA的缺点

Disadvantages of SQA

- ❑ It is difficult to introduce in small organizations, where available resources to perform necessary activities are not available
- ❑ It represents cultural change - and change is never easy
- ❑ It requires the expenditure of dollars that would not otherwise be explicitly budgeted to software engineering or QA
- ❑ 在小型组织中推行这一做法很困难，因为缺乏执行必要活动的可用资源
- ❑ 它代表了文化变革——而改变从来都不容易
- ❑ 这需要投入本不该明确用于软件工程或质量保证的资金

欢迎与会议概述

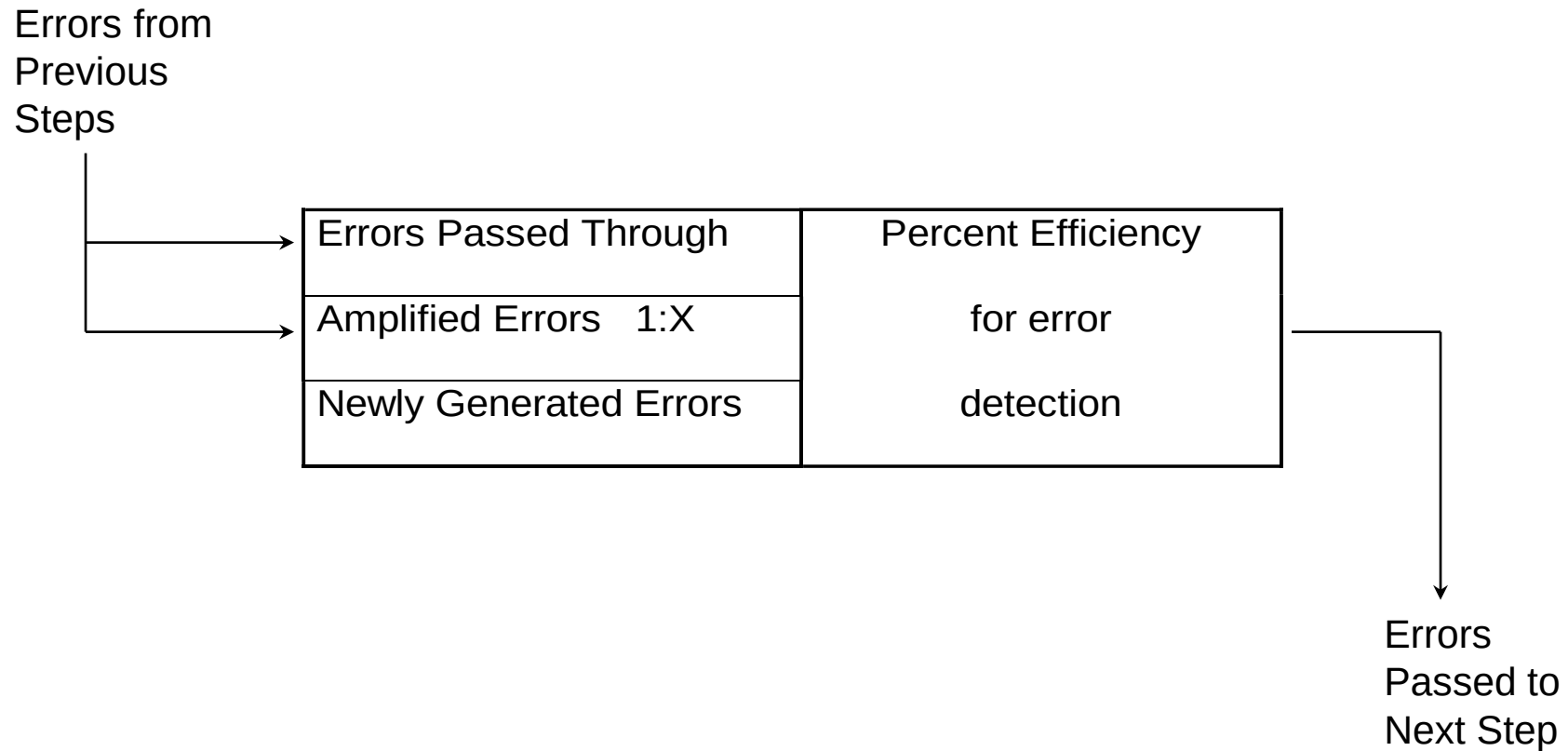
Quality Reviews



[Adapted from Pressman 4th Ed]

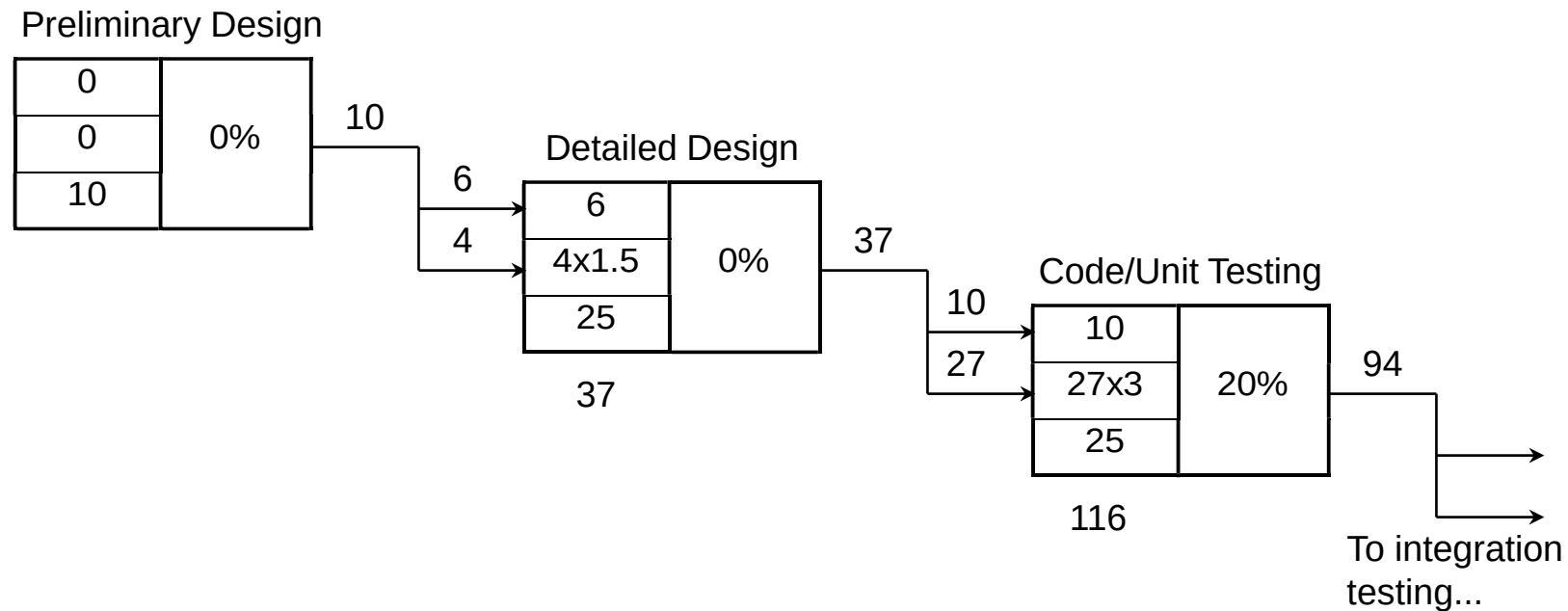
软件缺陷的成本影响

Cost Impact of Software Defects



缺陷放大与去除

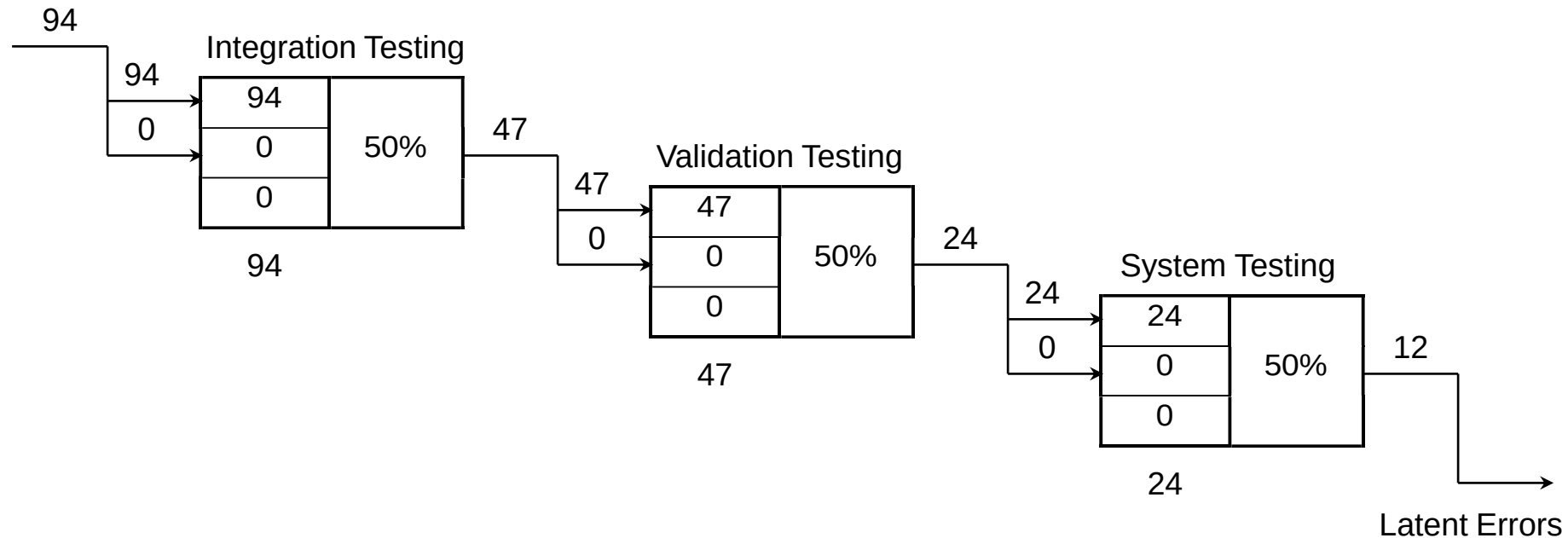
Defect Amplification and Removal



缺陷放大与去除

Defect Amplification and Removal

Defect Amplification (cont'd)



软件项目规划复习清单

Review Checklist for Software Project Planning

- ☐ Is the software scope unambiguously defined and bounded?
- ☐ Is terminology clear?
- ☐ Are resources adequate for the scope?
- ☐ Are resources readily available?
- ☐ Are tasks properly defined and sequenced?
- ☐ Is the basis for cost estimation reasonable? Has it been developed using two different sources?
- ☐ Have historical productivity and quality data been used?
- ☐ Have differences in estimates been reconciled?
- ☐ Are pre-established budgets and deadlines realistic?
- ☐ Is the schedule consistent?
- ☐ 软件范围是否明确且有界限?
- ☐ 术语清晰吗?
- ☐ 资源是否足够覆盖范围?
- ☐ 资源是否随手可得?
- ☐ 任务是否被正确定义和排序?
- ☐ 成本估算的基础合理吗? 它是用两个不同的资料开发的吗?
- ☐ 是否使用了历史生产力和质量数据?
- ☐ 估计差异是否已被调和?
- ☐ 预设的预算和截止日期现实吗?
- ☐ 时间安排一致吗?

软件需求分析复习清单

Review Checklist for Software Requirements Analysis

- ☐ Is the information domain analysis complete, consistent, and accurate?
- ☐ Is problem partitioning complete?
- ☐ Are external and internal interfaces properly defined?
- ☐ Are all requirements traceable to the system level?
- ☐ Is prototyping conducted for the customer?
- ☐ Is performance achievable with constraints imposed by other system elements?
- ☐ Are requirements consistent with schedule, resources, and budget?
- ☐ Are validation criteria complete?
- ☐ 信息领域分析是否完整、一致且准确?
- ☐ 问题分区完成了吗?
- ☐ 外部和内部接口定义得当吗?
- ☐ 所有需求都能追溯到系统层面吗?
- ☐ 原型制作会为客户进行吗?
- ☐ 在其他系统元素施加的约束下, 性能是否可以实现?
- ☐ 需求是否与进度、资源和预算一致?
- ☐ 验证标准完整吗?

软件设计复习清单（初步设计评审）

Review Checklist for Software Design(Preliminary Design Review)

- ☐ Are software requirements reflected in the software architecture?
- ☐ Is effective modularity achieved? Are modules functionally independent?
- ☐ Is program architecture factored?
- ☐ Are interfaces defined for modules and external system elements?
- ☐ Is data structure consistent with software requirements?
- ☐ Has maintainability been considered?
- ☐ 软件需求是否反映在软件架构中?
- ☐ 有效的模块化是否实现? 模块在功能上是独立的吗?
- ☐ 程序架构会被分解吗?
- ☐ 模块和外部系统元素是否定义了接口?
- ☐ 数据结构是否符合软件需求?
- ☐ 有没有考虑过可维护性?

软件设计复习清单 (设计攻略)

Review Checklist for Software Design (Design Walkthrough)

- ☐ Does the algorithm accomplish the desired function?
- ☐ Is the algorithm logically correct?
- ☐ Is the interface consistent with architectural design?
- ☐ Is logical complexity reasonable?
- ☐ Have error handling and “antibugging” been specified?
- ☐ Is local data structure properly defined?
- ☐ Are structured programming constructs used throughout?
- ☐ Is design detail amenable to the implementation language?
- ☐ Which are used: operating system or language dependent features?
- ☐ Is compound or inverse logic used?
- ☐ Has maintainability been considered?
- ☐ 算法是否实现了预期的功能?
- ☐ 算法逻辑正确吗?
- ☐ 界面是否与建筑设计一致?
- ☐ 逻辑复杂度合理吗?
- ☐ 错误处理和“防虫”功能有明确说明吗?
- ☐ 本地数据结构定义得当吗?
- ☐ 结构化编程结构贯穿整个过程吗?
- ☐ 设计细节适合实现语言吗?
- ☐ 使用了哪些: 操作系统或语言相关功能?
- ☐ 复合逻辑还是逆逻辑?
- ☐ 有没有考虑过可维护性?

编程复习清单

Review Checklist for Coding

- ☐ Is the design properly translated into code? (The results of the procedural design should be available at this review)
- ☐ Are there misspellings or typos?
- ☐ Has proper use of language conventions been made?
- ☐ Is there compliance with coding standards for language style, comments?
- ☐ Examples of Camel Case
- ☐ Here is an some examples of how camel case:
 - ☐ numberOfDonuts
- ☐ Are incorrect or ambiguous comments present?
- ☐ Are typing and data declaration proper?
- ☐ Are physical constraints correct?
- ☐ Have all items on the design walkthrough checklist been reapplied (as required)?
- ☐ 设计是否被正确转化成代码? (程序设计结果应在本综述中获得)
- ☐ 有拼写错误或拼写错误吗?
- ☐ 语言惯例的正确使用是否被规范?
- ☐ 语言风格、注释、模块序言是否符合编码标准?
- ☐ 有不正确或含糊的评论吗?
- ☐ 打字和数据声明合适吗?
- ☐ 物理约束正确吗?
- ☐ 设计流程清单上的所有项目是否都按要求重新应用了?

软件测试复习清单（测试计划）

Review Checklist for Software Testing (Test Plan)

- ☐ Have major test phases been properly identified and sequenced?
- ☐ Has traceability to validation criteria/requirements been established as part of software requirements analysis?
- ☐ Are major functions demonstrated early?
- ☐ Is the test plan consistent with the overall project plan?
- ☐ Has a test schedule been explicitly defined?
- ☐ Are test resources and tools identified and available?
- ☐ Has a test recordkeeping mechanism been established?
- ☐ Have test drivers and stubs been identified, and has work to develop them been scheduled?
- ☐ Has stress testing for software been specified?
- ☐ 主要测试阶段是否已被正确识别和排序?
- ☐ 作为软件需求分析的一部分，是否已经建立了验证标准/要求的可追溯性?
- ☐ 主要功能是否会早期展示?
- ☐ 测试计划与整体项目计划一致吗?
- ☐ 考试安排是否被明确定义?
- ☐ 测试资源和工具是否被识别并可用?
- ☐ 是否建立了测试记录机制?
- ☐ 测试车手和存根已经确定了吗？并安排了相关工作?
- ☐ 软件的压力测试有被规定吗?

软件测试复习清单（测试程序）

Review Checklist for Software Testing(Test Procedure)

- ☐ Have both white and black box tests been specified?
- ☐ Have all independent logic paths been tested?
- ☐ Have test cases been identified and listed with expected results?
- ☐ Is error handling to be tested?
- ☐ Are boundary values to be tested?
- ☐ Are timing and performance to be tested?
- ☐ Has acceptable variation from expected results been specified?
- ☐ 白盒和黑盒测试都被指定了吗?
- ☐ 所有独立逻辑路径都测试过了吗?
- ☐ 测试案例是否已被识别并列出预期结果?
- ☐ 错误处理需要被测试吗?
- ☐ 边界值需要测试吗?
- ☐ 需要测试时机和性能吗?
- ☐ 是否明确了与预期结果的可接受差异?

维护复习清单

Review Checklist for Maintenance

- ☐ Have side effects associated with change been considered?
- ☐ Has the request for change been documented, evaluated, and approved?
- ☐ Has the change, once made, been documented and reported to interested parties?
- ☐ Have appropriate FTRs been conducted?
- ☐ Has a final acceptance review been conducted to assure that all software has been properly updated, tested, and replaced?
- ☐ 有没有考虑过与变化相关的副作用?
- ☐ 变更请求是否已被记录、评估并批准?
- ☐ 变更一旦发生, 是否已被记录并报告给相关方?
- ☐ 是否进行了适当的FTR检查?
- ☐ 是否进行了最终验收审查, 以确保所有软件均已正确更新、测试和更换?

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Why Quality Matters

"From Frustration to Business Failure"

Connect bugs to business / societal consequences:

- ☐ Lost revenue
- ☐ Reputational damage
- ☐ Safety-critical failures
- ☐ Legal and compliance risks

Key message: Quality is not just about testing — it is a strategic imperative.

为什么质量很重要

《从挫败到商业失败》

将漏洞与商业/社会后果联系起来：

- ☐ 收入损失
- ☐ 声誉损害
- ☐ 安全关键故障
- ☐ 法律与合规风险

关键信息：质量不仅仅是测试——它是一种战略上的必然。

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

What Is Software Quality?

Three classic perspectives:

- ☐ Conformance to requirements (Crosby)
- ☐ Fitness for use (Juran)
- ☐ Value to stakeholders (modern view)

Transition: To manage quality, we must be specific about what attributes we care about.

什么是软件质量？

三种经典视角：

- ☐ 符合要求（克罗斯比）
- ☐ 适用性（Juran）
- ☐ 对利益相关者的价值（现代视角）

过渡：要管理质量，我们必须明确关注哪些属性。

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

ISO 25010 Product Quality Model

Characteristics listed:

1. Functional Suitability
2. Reliability
3. Usability
4. Performance Efficiency
5. Maintainability
6. Portability
7. Security
8. Compatibility

ISO 25010 产品质量模型

列出的特征：

1. 功能适用性
2. 可靠性
3. 可用性
4. 性能效率
5. 可维护性
6. 便携性
7. 安全性
8. 兼容性

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Quality Attributes in Action

1. **Functional Suitability:** The system does what it's supposed to do (e.g., a calculator returns correct results)
2. **Reliability:** System operates without failure under specified conditions (e.g., banking app works during peak hours)
3. **Usability:** Easy to learn, efficient to use (e.g., intuitive navigation on an e-commerce site)
4. **Performance Efficiency:** Response time, resource usage (e.g., search results within 2 seconds)

质量属性的实际应用

1. **功能适用性：**系统能够完成其应有的功能（例如，计算器返回正确结果）
2. **可靠性：**系统在特定条件下无故障运行（例如银行应用在高峰时段正常工作）
3. **可用性：**易于学习，使用高效（例如，电子商务网站的直观导航）
4. **性能效率：**响应时间、资源使用（例如，搜索结果在2秒内）

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

- 5. **Maintainability:** How easily can the system be modified (e.g., adding a new payment method without breaking existing code)
 - 6. **Portability:** Can the system run in different environments (e.g., cloud-to-cloud migration, mobile OS versions)
 - 7. **Security:** Protection of data and system access (e.g., encrypted user data, two-factor authentication)
 - 8. **Compatibility:** Coexistence and interoperability with other systems (e.g., a plugin works in different browsers)
- 5. **可维护性:** 系统修改的难易程度 (例如, 添加新的支付方式而不破坏现有代码)
 - 6. **可移植性:** 系统能否在不同环境中运行 (例如云到云迁移、移动操作系统版本)
 - 7. **安全性:** 保护数据和系统访问 (例如, 加密用户数据、双因素认证)
 - 8. **兼容性:** 与其他系统共存和互操作性 (例如, 插件可在不同浏览器中运行)

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Quality's Big Three: QA, QC, QM

- ❑ Common confusion in industry
- ❑ Quick definitions (advance organizer):
 - ❑ **Quality Assurance (QA)**: Process-oriented, defect prevention
 - ❑ **Quality Control (QC)**: Product-oriented, defect detection
 - ❑ **Quality Management (QM)**: Organizational framework for quality

质量的三大关键环节：QA, QC, QM

- ❑ 工业界常见的混淆
- ❑ 快速定义（提前组织）：
 - ❑ **质量保证（QA）**：以流程为导向，防止缺陷
 - ❑ **质量控制（QC）**：面向产品，缺陷检测
 - ❑ **质量管理（QM）**：质量的组织框架

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

QA vs QC: Prevention vs Detection

QA (Quality Assurance):

- ☐ Focus: The process
- ☐ Activities: Audits, reviews, training, process definition
- ☐ Goal: Build it right the first time

QC (Quality Control):

- ☐ Focus: The product
- ☐ Activities: Testing, inspection, measurement
- ☐ Goal: Find defects before release

QA与QC：预防与检测

质量保证（QA）：

- ☐ 重点：过程
- ☐ 活动内容：审计、审查、培训、流程定义
- ☐ 目标：第一次就做好

QC（质量控制）：

- ☐ 重点：产品本身
- ☐ 活动：测试、检查、测量
- ☐ 目标：在发布前发现缺陷

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Quality Management (QM) — The Big Picture

QM coordinates QA and QC

Includes:

- ☐ Quality planning
- ☐ Quality policies and objectives
- ☐ Resource allocation
- ☐ Continuous improvement
- ☐ QM ensures QA and QC are aligned and sustainable.

质量管理（QM）—大局

QM坐QA和QC

内容包括：

- ☐ 质量规划
- ☐ 质量政策与目标
- ☐ 资源分配
- ☐ 持续改进
- ☐ QM确保QA和QC保持一致且可持续。

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Example from Construction

- ❑ **QA:** Building codes, foundation inspection during the pour
- ❑ **QC:** Checking finished walls for cracks, testing door alignment
- ❑ **QM:** The project manager's quality plan, contractor certifications, inspection schedules

Reinforcement: QA is your construction standards, QC is your finished inspection, QM is your whole quality system.

构造示例

- ❑ **QA:** 建筑规范，浇筑时的地基检查
- ❑ **QC:** 检查完工墙体是否有裂缝，测试门的对齐
- ❑ **QM:** 项目经理的质量计划、承包商认证、检查计划

强化：QA是你的施工标准，QC是你的最终验收，质量管理是你的整个质量体系。

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Apply the analogy to software:

QA: Code review checklists, automated static analysis setup, definition of done

QC: Unit test execution, manual testing, user acceptance testing

QM: Test strategy document, quality gates in CI/CD, quality objectives and KPIs

将这个类比应用到软件上：

QA: 代码审查检查表、自动静态分析设置、完成定义

QC: 单元测试执行、手动测试、用户验收测试

QM: 测试策略文档、CI/CD中的质量门、质量目标和关键绩效指标（KPI）

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

The Cost of Quality (CoQ)

Definition: The total cost of ensuring quality (conformance) plus the cost of failures (non-conformance)

Understanding CoQ helps justify investment in QA activities

Key message: Prevention is almost always cheaper than fixing defects later.

质量成本 (CoQ)

定义：确保质量（符合性）的总成本加上失效成本（不符合性）

理解CoQ有助于为QA活动的投资提供合理性

关键信息：预防几乎总是比以后修复缺陷更便宜。

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

The Four Categories of CoQ

Conformance costs:

1. **Prevention:** Training, process improvements, early reviews
2. **Appraisal:** Testing, inspections, audits

Non-conformance costs:

3. **Internal Failure:** Defects found before release (rework, re-testing)
4. **External Failure:** Defects found after release (hotfixes, support calls, lawsuits, reputation loss)

CoQ的四类

合规成本:

1. **预防:** 培训、流程改进、早期审查
2. **评估:** 测试、检查、审计

不合规成本:

3. **内部故障:** 发布前发现缺陷（重做、重新测试）
4. **外部故障:** 发布后发现的缺陷（热修复、支持电话、诉讼、声誉损失）

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Defects Get More Expensive Over Time

Cost Escalation – The 1:10:100 Rule

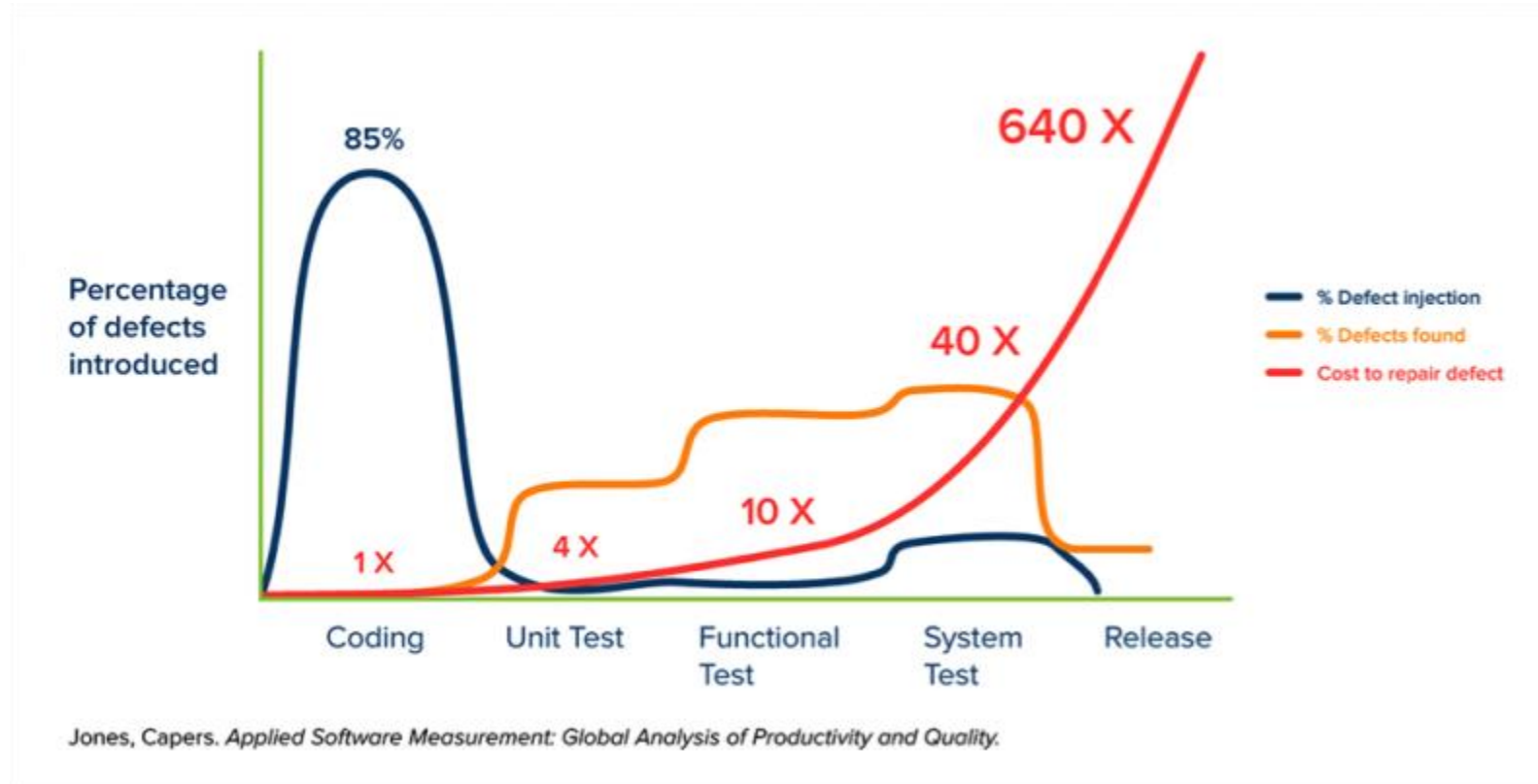


基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Defects Get More Expensive Over Time

Cost Escalation – The 1:10:100 Rule



Week 1: Introduction to Software Quality Assurance & Management 第1周：软件质量保证与管理导论

北京工业大学 软件质量保证与管理

Beijing University of Technology, Software Quality Assurance and Management

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Where Does Each Activity Belong?

- | | |
|--|----------------------|
| A. Writing unit tests before coding | A – Prevention |
| B. Manual regression testing before release | B – Appraisal |
| C. Fixing a bug found during a sprint | C – Internal Failure |
| D. A production outage requiring an emergency hotfix | D – External Failure |
| E. Peer code review | E – Prevention |
| F. Penetration testing by an external team | F – Appraisal |

Instruction: Classify each as Prevention, Appraisal, Internal Failure, or External Failure.

Borderline items (e.g., a code review could be seen as appraisal if just finding defects, but it's usually prevention because it fixes issues early).

基础：质量概念与质量的成本

Foundations: Quality Concepts and the Cost of Quality

Recap

- ❑ Software quality is multi-dimensional (ISO 25010)
- ❑ QA (process/prevent), QC (product/detect), QM (management system)
- ❑ Cost of Quality: Prevention < Appraisal < Internal Failure << External Failure
- ❑ Investing in quality processes reduces overall cost and risk

回顾

- ❑ 软件质量具有多维度 (ISO 25010)
- ❑ QA (过程/预防)、QC (产品/检测)、QM (管理系统)
- ❑ 质量成本：内部失效<外部失效<预防<<评估
- ❑ 投资于高质量流程可降低整体成本和风险

生命周期质量

Quality Across Lifecycles

Does quality happen only at the end, or is it built throughout?

质量是只在结尾出现，还是贯穿始终？

We will examine how Waterfall, V-Model, and Spiral models embed (or fail to embed) quality activities.

我们将探讨瀑布模型、V模型和螺旋模型如何嵌入（或未能嵌入）优质活动。

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Quick Recap & Today's Agenda

- ❑ Quality attributes (ISO 25010)
- ❑ QA (prevention) vs QC (detection) vs QM (management)
- ❑ Cost of Quality: Prevention < Appraisal < Internal Failure < External Failure

Next agenda:

- ❑ The SDLC and quality timing
- ❑ Waterfall model
- ❑ V-Model
- ❑ Spiral model
- ❑ Comparison and key takeaways

快速回顾与今日议程

- ❑ 质量属性 (ISO 25010)
- ❑ QA (预防) VS QC (检测) VS QM (管理)
- ❑ 质量成本: 内部失效<外部失效<预防<评估

下一个议程:

- ❑ SDLC与质量时机
- ❑ 瀑布模型
- ❑ V型
- ❑ 螺旋模型
- ❑ 比较与主要结论

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

The SDLC Dictates Quality Timing

OR

"The Lifecycle Shapes the Quality Approach"

Core idea: When and how we apply quality activities depends on the development lifecycle model.

In traditional, plan-driven models, quality is structured into phases.

Common problem: testing and final validation are left until late stages.

This session explores how Waterfall, V-Model, and Spiral each embed quality.

SDLC规定了高质量的时机

或者

“生命周期塑造质量方法”

核心理念：我们何时以及如何应用质量活动取决于开发生命周期模型。

在传统的计划驱动模型中，质量被结构化多个阶段。

常见问题：测试和最终验证要拖到后期。

本节将探讨Waterfall、V-Model和Spiral如何各自嵌入质量。

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

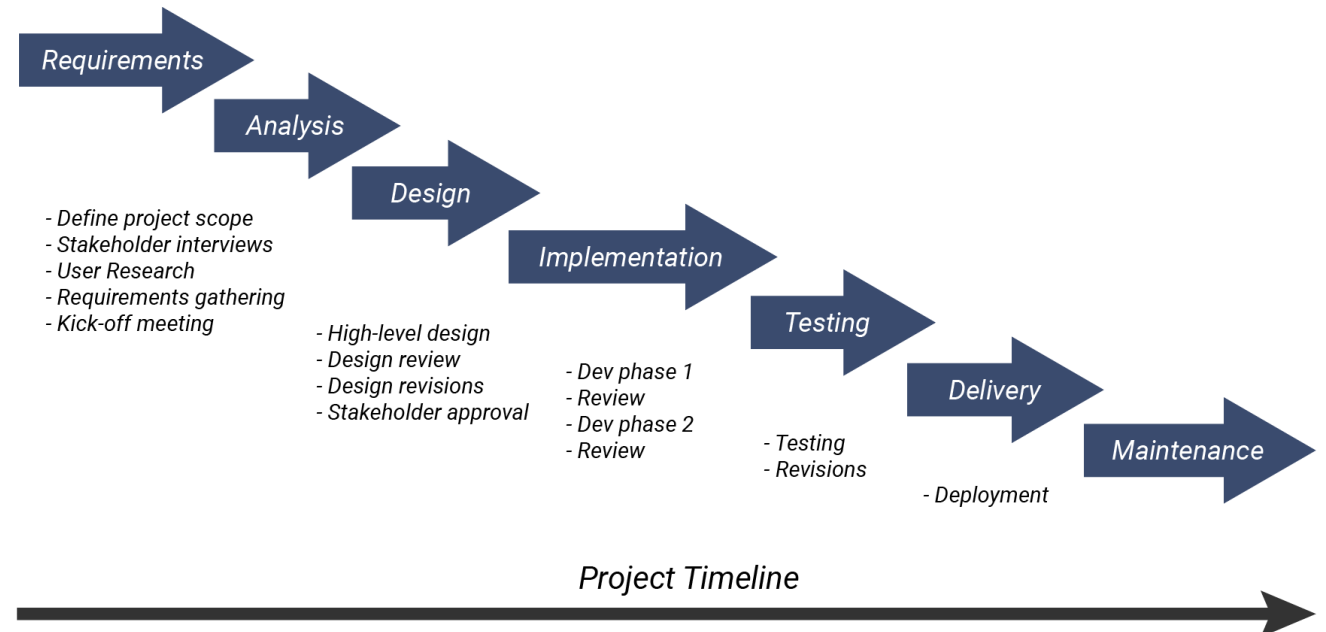
Waterfall Model – A Sequential Approach

Key characteristics:

- ☐ Each phase is completed and approved before the next begins.
- ☐ Heavy documentation and sign-offs.
- ☐ Testing is a distinct, late phase.

Waterfall

(Plan Driven)



传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

QA & QC Activities in Waterfall

Overlay on phases:

- ❑ **Requirements phase:** reviews, inspections (QA – prevention)
- ❑ **Design phase:** design reviews, architecture inspections (QA)
- ❑ **Implementation:** code reviews, static analysis (QA/QC)
- ❑ **Testing phase:** execution of test cases, defect reporting (QC)
- ❑ **Maintenance:** regression testing, patches (ongoing QC)

Emphasis: most formal testing occurs only after code is complete.

QA和QC瀑布地区的活动

相位叠加:

- ❑ 需求阶段：审查、检查（质量保证——预防）
- ❑ 设计阶段：设计审查，架构检查（QA）
- ❑ 实施：代码审查，静态分析（质量保证/质量控制）
- ❑ 测试阶段：测试用例执行，缺陷报告（QC）
- ❑ 维护：回归测试，补丁（持续质检）

重点是：大多数正式测试只在代码完成后进行。

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

The "Testing at the End" Problem

But

"Late Discovery, High Cost"

Issue: Defects introduced in early phases (e.g., misunderstood requirements) remain hidden until the testing phase.

Consequences:

- ☐ Expensive rework
- ☐ Schedule delays
- ☐ Pressure to skip thorough testing

“最后测试”问题

但是

“迟发现，代价高昂”

问题：早期阶段引入的缺陷（例如被误解的需求）会隐藏到测试阶段。

后果：

- ☐ 昂贵的重做
- ☐ 赛程延误
- ☐ 被迫跳过严格检测的压力

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Waterfall – Quality Strengths & Weaknesses

Strengths:

- ☐ Clear phase gates and documentation
- ☐ Formal review stages possible
- ☐ Mature, well-understood model

Weaknesses:

- ☐ Late feedback on product functionality
- ☐ Inflexible to changing requirements
- ☐ High risk of significant quality escapes if testing phase is compressed

瀑布—质量优势与劣势

优势:

- ☐ 清晰的相位门和文档
- ☐ 可能有正式审查阶段
- ☐ 成熟且被广泛理解的模型

弱点:

- ☐ 关于产品功能的反馈迟到。
- ☐ 对不断变化的要求不灵活
- ☐ 如果测试阶段被压缩，质量有显著泄漏的风险

传统生命周期模型中的质量

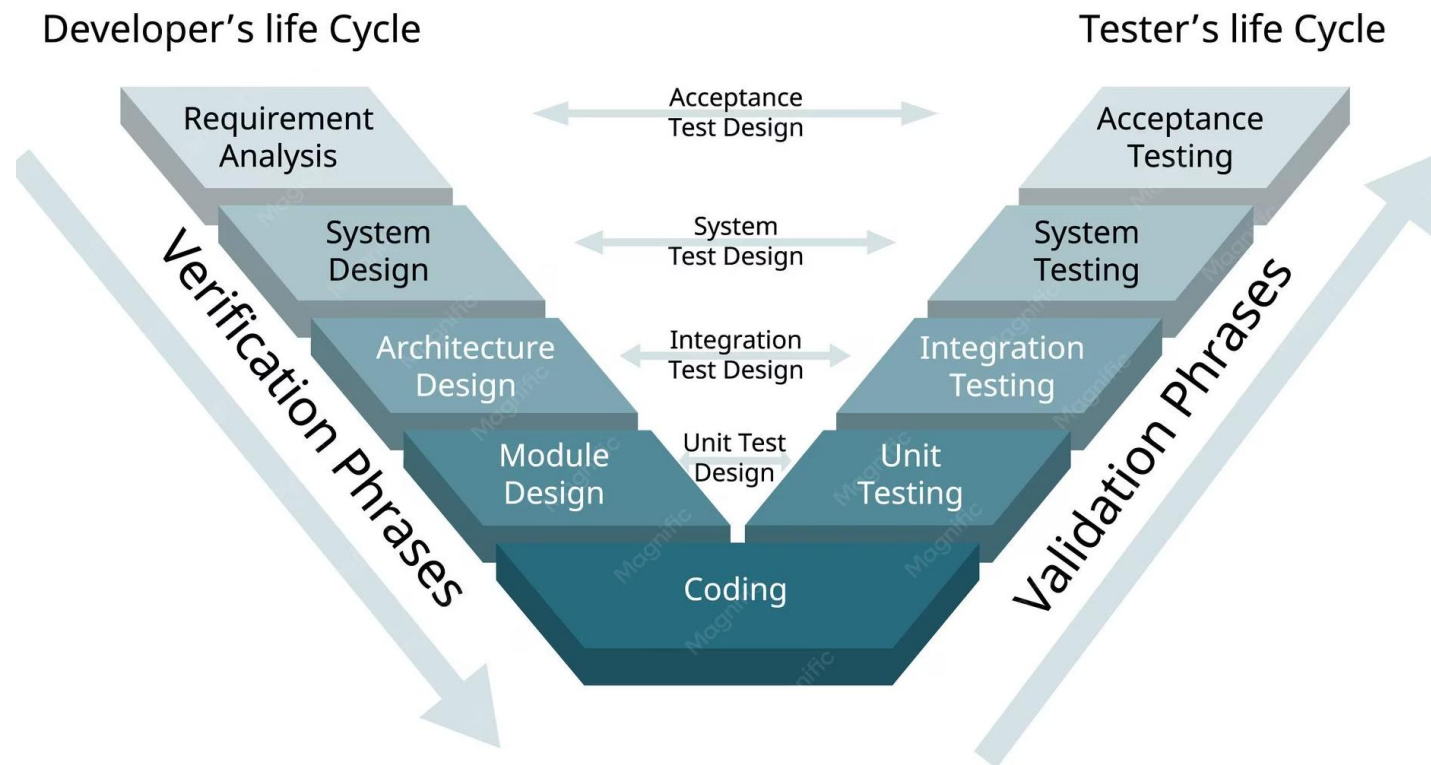
Quality in Traditional Lifecycle Models

V-Model – Linking Verification and Validation

OR

"The V-Model: Bringing Testing Forward"

Key concept: Each development phase has a corresponding test phase; test planning happens alongside development.



传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Verification and Validation in the V-Model

Left side (development) and right side (testing):

- ❑ Requirements ↔ Acceptance Testing
- ❑ High-Level Design ↔ System Testing
- ❑ Detailed Design ↔ Integration Testing
- ❑ Coding ↔ Unit Testing

Verification: "Are we building the product right?"
(reviews, unit tests)

Validation: "Are we building the right product?"
(system/acceptance tests)

V模型中的验证与确认

左侧（开发）和右侧（测试）：

- ❑ 需求↔验收测试
- ❑ 高层设计↔系统测试
- ❑ 详细设计↔集成测试
- ❑ 编码↔单元测试

验证：“我们做产品做对了吗？”（评测，单元测试）

验证：“我们正在打造正确的产品吗？”（系统/验收测试）

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Early Test Planning Advantage

In the V-Model, acceptance test cases can be derived from requirements, integration tests from design, etc., while coding is ongoing.

Benefits:

- ☐ Ambiguities in requirements found before coding
- ☐ Clear traceability from requirements to tests
- ☐ Reduces the "testing crunch" at the end

早期测试规划优势

在V模型中，验收测试用例可以从需求中推导出，集成测试可以从设计中推导出来，而编码则是持续进行的。

福利：

- ☐ 编码前发现的需求歧义
- ☐ 从需求到测试的明确可追溯性
- ☐ 减少了最后的“测试紧张”

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

V-Model – Quality Strengths & Weaknesses

Strengths:

- ☐ Explicit verification and validation links
- ☐ Early test planning and design
- ☐ Strong traceability

Weaknesses:

- ☐ Still sequential and rigid
- ☐ Late integration testing can still surface costly defects
- ☐ Does not easily accommodate changing requirements

V模型—质量优势与劣势

优势:

- ☐ 显式验证与确认链接
- ☐ 早期测试规划与设计
- ☐ 强可追溯性

弱点:

- ☐ 依然顺序且刚性
- ☐ 晚期集成测试仍可能暴露出代价高昂的缺陷
- ☐ 不易满足不断变化的需求

传统生命周期模型中的质量

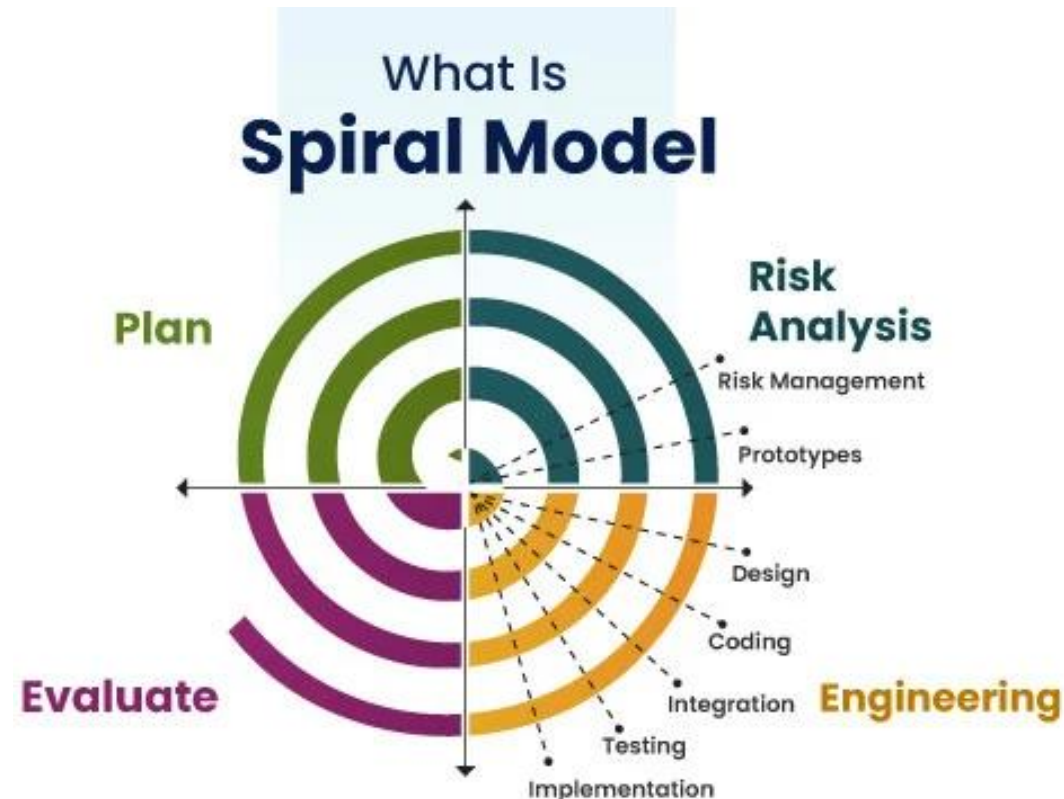
Quality in Traditional Lifecycle Models

Spiral Model – Risk-Driven Iterations

Visual: spiral diagram showing iterative cycles, each with four quadrants:

1. Determine objectives
2. Identify and resolve risks
3. Develop and test
4. Plan next iteration

Risk analysis is the central activity driving scope, prototyping, and testing.



传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Quality as a Risk Mitigation Strategy

In each spiral loop, quality activities are chosen based on identified risks.

Examples:

- ❑ High performance risk → early performance modeling and testing
- ❑ Requirements uncertainty → **heavy prototyping** and user reviews
- ❑ Integration risk → frequent builds and partial integration testing

QA becomes proactive and tailored, not a one-size-fits-all phase.



传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Spiral Model – Quality Strengths & Weaknesses

Strengths:

- ☐ Risk-driven testing and reviews
- ☐ Iterative feedback reduces late surprises
- ☐ Early identification of critical quality issues

Weaknesses:

- ☐ Can be complex to manage
- ☐ Still larger cycles compared to Agile sprints
- ☐ Quality depends heavily on accurate risk identification

螺旋模型—质量优势与劣势

优势:

- ☐ 风险驱动测试与评审
- ☐ 迭代反馈减少了晚期的意外
- ☐ 早期识别关键质量问题

弱点:

- ☐ 管理起来可能很复杂
- ☐ 相比敏捷冲刺，这些周期仍然更大
- ☐ 质量高度依赖于准确的风险识别

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Feature	Waterfall	V-Model	Spiral
Timing of Testing	Late; testing only begins after the entire implementation phase is complete.	Earlier planning; test planning happens alongside requirements and design, though execution is still late.	Iterative; testing and evaluation happen continuously at the end of each cycle.
Feedback Loops	Weak; feedback flows rigidly to the next phase, making it hard to revisit previous steps.	Weak; while structured, the rigid verification and validation sequence makes feedback slow to implement.	Medium; feedback is gathered iteratively after each prototype or cycle, allowing for course correction.
Flexibility to Change	Low; highly rigid, and changes are costly once a phase is signed off.	Low; changes require revisiting the entire corresponding verification and validation flow.	High; accommodates changes easily as requirements can be adjusted in subsequent iterations.
Quality Focus	Document-driven phase sign-offs; assumes quality if requirements are met linearly.	Verification and Validation; ensures every development phase has a corresponding testing phase.	Risk-focused; prioritizes identifying and mitigating risks early through continuous prototyping and evaluation.

传统生命周期模型中的质量

Quality in Traditional Lifecycle Models

Key Takeaways for Quality Managers

- ❑ The earlier we validate, the cheaper quality becomes (V-Model improves on Waterfall).
- ❑ Risk-based thinking should drive quality effort (Spiral).
- ❑ Sequential models still suffer from delayed integration and feedback.
- ❑ These limitations led to the rise of Agile and DevOps.

质量管理者的关键点

- ❑ 验证越早，质量越便宜（V-Model 优于 Waterfall）。
- ❑ 基于风险的思维应推动高质量的努力（螺旋）。
- ❑ 顺序模型仍然存在整合延迟和反馈问题。
- ❑ 这些限制促使了敏捷和DevOps的兴起。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Quick Recap & Today's Agenda

- ❑ Waterfall: testing at the end, expensive late defects
- ❑ V-Model: verification/validation pairs, earlier test planning
- ❑ Spiral: risk-driven iterations, tailored quality activities
- ❑ Common limitation: sequential and slow feedback

This Session:

- ❑ Warm-up: can we release fast and keep quality?
- ❑ Agile quality: shift-left, built-in quality
- ❑ DevOps quality: continuous testing and monitoring
- ❑ Comparison: traditional vs Agile vs DevOps
- ❑ Key takeaways

快速回顾与今日议程

- ❑ 瀑布式：最后测试，昂贵的后期缺陷
- ❑ V模型：验证/确认对，早期测试规划
- ❑ 螺旋：风险驱动的迭代，量身定制的高质量活动
- ❑ 常见限制：顺序反馈和慢反馈

本场次：

- ❑ 热身：我们能否快速发布并保持质量？
- ❑ 敏捷品质：左移，内置品质
- ❑ DevOps 质量：持续测试与监控
- ❑ 对比：传统、敏捷与DevOps
- ❑ 主要要点

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Releasing Every Two Weeks – Is Quality at Risk?

Prompt: If your team releases software every two weeks, what are the biggest quality risks you face?

Common fears:

- ☐ “Not enough time to test”
- ☐ “Regressions slip through”
- ☐ “Integration breaks in production”

Transition: Agile and DevOps exist to solve these exact problems—not by cutting quality, but by embedding it.

每两周发布一次—质量是否存在风险？

提示：如果你们团队每两周发布一次软件，面临的**最大质量风险**是什么？

常见的担忧：

- ☐ “没时间测试”
- ☐ “回溯会穿过”
- ☐ “生产中的整合中断”

转型：敏捷和DevOps的存在正是为了解决这些问题—不是通过降低质量，而是通过将质量嵌入。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

The Shift-Left Principle

OR

“Shift-Left: Move Quality Upstream”

Visual: Timeline arrow from “Requirements” to “Production”, with traditional testing concentrated on the right. A second arrow shows testing activities starting earlier (shift-left).

Definition: Shifting testing, reviews, and quality checks to earlier in the development cycle.

Goal: Prevent defects, don't just find them later.

左移原则

或者

“左移：向上游移动质量”

可视化：时间线箭头从“需求”到“生产”，传统测试集中在右侧。第二个箭头显示测试活动开始得更早（向左移位）。

定义：将测试、审查和质量检查提前到开发周期的早期阶段。

目标：预防缺陷，而不是事后才发现。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Agile Mindset – Built-In Quality

- ❑ **Core Agile principle:** Quality is never traded for speed; it's integral to every increment.
- ❑ **Continuous feedback from multiple sources:** developers, testers, product owner, users.
- ❑ Short iterations force quality practices to be fast and effective.
- ❑ Testing is not a phase—it's an ongoing activity within each sprint.

敏捷思维—内建质量

- ❑ 敏捷核心原则：质量绝不被速度所取代;它是每一个增量的组成部分。
- ❑ 来自多个来源的持续反馈：开发者、测试人员、产品负责人、用户。
- ❑ 短迭代迫使质量实践快速且高效。
- ❑ 测试不是一个阶段—它是每个冲刺中持续进行的活动。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Quality in Scrum Practices

Key Scrum elements that embed quality:

- ❑ Definition of Done (DoD): includes test completion, code review, no open defects
- ❑ Sprint Planning: quality tasks estimated and allocated
- ❑ Daily Standup: blockers and test progress visible
- ❑ Sprint Review: stakeholder feedback on working software
- ❑ Sprint Retrospective: continuous improvement of quality processes

Whole team owns quality—not just a “tester.”

Scrum 实践中的质量

关键的Scrum核心要素是提升质量的：

- ❑ 完成定义（DoD）：包括测试完成、代码审查、无未解决缺陷
- ❑ 冲刺规划：质量任务的估算与分配
- ❑ 每日站立：阻碍因素和测试进展可见
- ❑ Sprint 评审：利益相关者对可用软件的反馈
- ❑ 冲刺回顾：质量流程的持续改进

整个团队都拥有质量—不仅仅是“测试员”。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

DEFINITION OF DONE

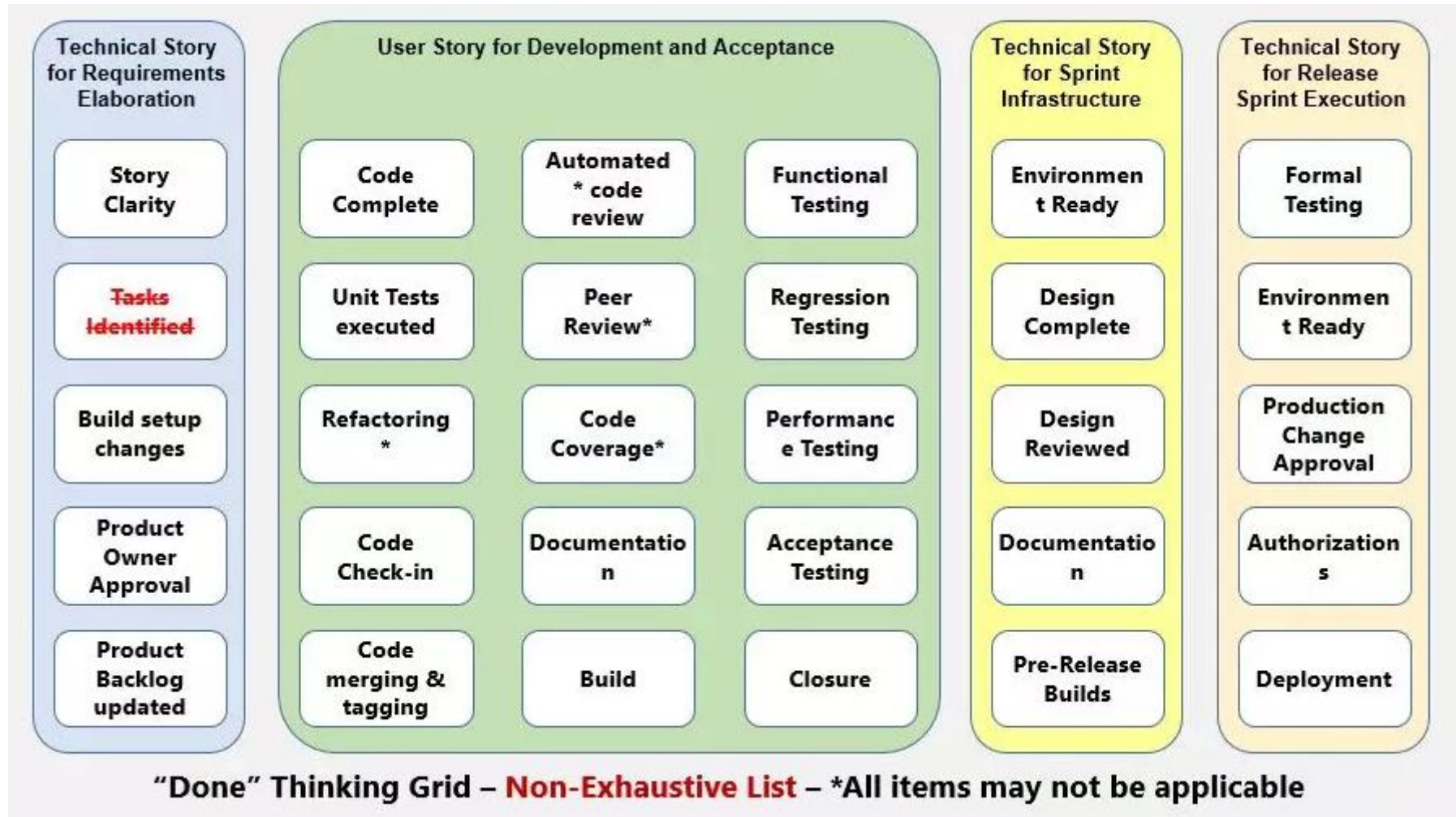
Team: _____
Product: _____

As a team, before saying that an item of the sprint backlog is Done, we agree that it will meet the following:

- ☒ Code is complete and according to development team standards.
- ☒ Code refactored.
- ☒ Meet acceptance criteria.
- ☐ Code checked-in to the repository.
- ☒ Unit test written and green.
- ☐ Test coverage: __ %.
- ☐ Pair programming.
- ☒ Peer review.
- ☐ Code merge and tagged.
- ☒ Deployed to the development environment.

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles



Week 1: Introduction to Software Quality Assurance & Management 第1周：软件质量保证与管理导论

北京工业大学 软件质量保证与管理

Beijing University of Technology, Software Quality Assurance and Management

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

TDD & BDD – Quality at the Code Level

TDD (Test-Driven Development):

- ❑ Write a failing test → write code to pass → refactor
- ❑ Guarantees test coverage and forces testable design

BDD (Behavior-Driven Development):

- ❑ Write executable specifications in “Given-When-Then” language
- ❑ Bridges business and technical understanding

Both are shift-left in action: defects prevented at the unit/acceptance level.

TDD 和 BDD – 代码层面的质量

TDD（测试驱动开发）：

- ❑ 写一个失败的测试→写代码通过重构→
- ❑ 保证测试覆盖率并强制设计可测试

BDD（行为驱动发展）：

- ❑ 用“给定-何时-那时”语言编写可执行规格
- ❑ 桥梁业务与技术理解

两者都是左移型的：在单位/验收层面防止缺陷。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Agile Quality – Summary

- ❑ Testing is continuous, not a final phase
- ❑ Whole-team responsibility (developers, testers, product owner)
- ❑ Heavy reliance on test automation for regression safety
- ❑ Frequent feedback loops (daily, per sprint)
- ❑ Quality is defined by working software validated by users

敏捷质量—总结

- ❑ 测试是连续进行的，而不是最终阶段
- ❑ 全团队责任（开发者、测试员、产品负责人）
- ❑ 回归安全高度依赖测试自动化
- ❑ 频繁的反馈循环（每天，每个冲刺）
- ❑ 质量由用户验证的工作软件定义

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Introduction to DevOps & Continuous Quality OR “From Agile to DevOps – Closing the Loop”

DevOps extends Agile principles to operations and production.

Goal: Shorten the feedback loop from “idea” to “working software in production” and back.

Quality must be embedded across development, delivery, and operations.

Key concept: “You build it, you run it” – quality responsibility extends to production.

DevOps 与持续质量介绍 或者 “从敏捷到DevOps—闭环”

DevOps 将敏捷原则扩展到运营和生产领域。

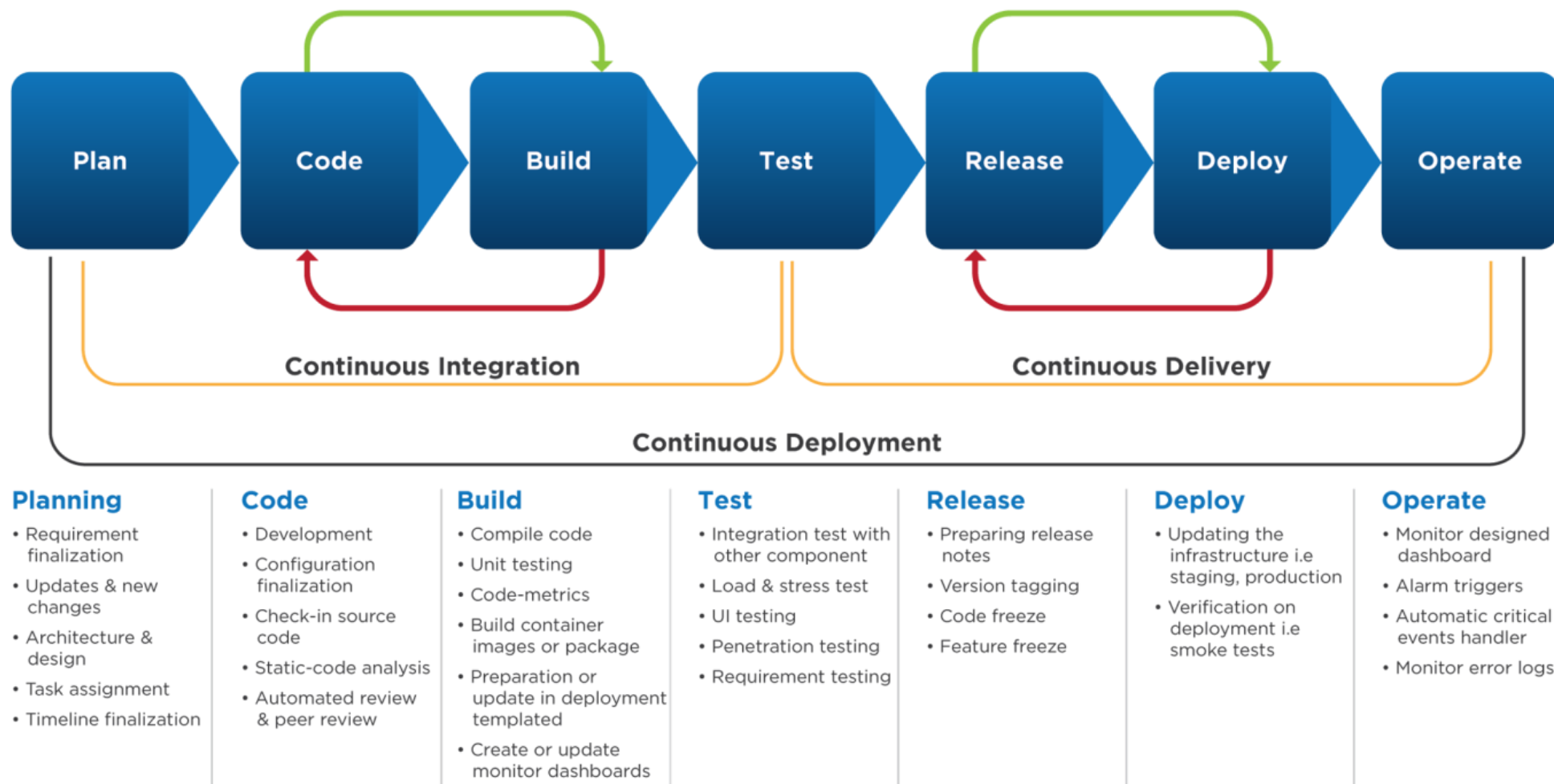
目标：缩短从“想法”到“生产中运行软件”的反馈循环。

质量必须贯穿开发、交付和运营。

关键概念：“你建造它，你运行它”——质量责任延伸到生产。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles



敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Quality Gates – Stop Bad Code Before It Escapes

Quality gates are automated checkpoints that prevent promotion if criteria are not met.

Examples:

- ❑ Unit test coverage < 80% → build fails
- ❑ Critical vulnerability found → deployment blocked
- ❑ Performance benchmark not met → rollback

Shift from manual approval to automated, data-driven go/no-go decisions.

优质门—在代码泄漏前阻止其

质量门槛是自动检查点，如果不符合条件，可以阻止晋升。

示例：

- ❑ 单元测试覆盖率<80%→构建失败
- ❑ 部署被阻断→发现关键漏洞
- ❑ 回滚→未达到性能基准

从人工审批转向自动化、数据驱动的可否决策。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Fail Fast, Recover Fast

Fail fast philosophy: detect defects at the earliest possible point in the pipeline.

Benefits:

- ☐ Cheaper fix (recall CoQ model)
- ☐ Faster learning
- ☐ Reduced deployment anxiety

cost/time graph showing defect cost increasing exponentially as it moves right in pipeline.

失败快，恢复快

快速失败的理念：在流程的最早点发现缺陷。

福利：

- ☐ 更便宜的修复方法（召回CoQ型号）
- ☐ 学习更快
- ☐ 减少部署焦虑

成本/时间图表显示缺陷成本随着流水线的增长呈指数增长。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Testing in Production – Observability & Chaos Engineering

Even with rigorous pre-production testing, production is the ultimate truth.

Monitoring and Observability:

- ❑ Logs, metrics, traces
- ❑ Real-time alerting on errors, performance degradation

Chaos Engineering:

- ❑ Deliberately injecting failures to test system resilience (e.g., Netflix Simian Army)

Canary releases and feature flags for safe rollouts.

生产环境中的测试——可观测性与混沌工程

即使经过严格的前期测试，生产才是最终的真相。

监控与可观察性：

日志、指标、痕迹
错误实时警报，性能下降

混沌工程：

故意注入失败内容来测试系统韧性（例如，Netflix 的《猿人军团》）

金丝雀发布和功能标志以确保安全部署。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

DevSecOps – Security as a Quality Attribute
OR
“Security Integrated from Day One”

DevSecOps embeds security into the DevOps pipeline.

Practices:

- ☐ Automated security scanning (SAST, DAST) in CI/CD
- ☐ Dependency vulnerability checks
- ☐ Infrastructure as Code scanning

Quality now explicitly includes security as a non-negotiable attribute.

DevSecOps – 安全作为质量属性
或者
“从第一天起就集成安全”

DevSecOps 将安全嵌入 DevOps 流水线。

练习:

- ☐ CI/CD 中的自动安全扫描 (SAST, DAST)
- ☐ 依赖漏洞检查
- ☐ 基础设施即代码扫描

质量现在明确将安全作为不可妥协的属性。

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

DIMENSION	TRADITIONAL (WATERFALL / V-MODEL)	AGILE (SCRUM / XP)	DEVOPS
TIMING OF TESTING	Testing at end, after implementation	Testing in every sprint, continuously throughout the iteration	Testing in the CI/CD pipeline and continuously in production
ROLE OF QA	Separate QA team, often siloed from development	Whole team shares quality responsibility; testers embedded in the team	Engineering owns quality; QA expertise shifted left and embedded in delivery
FEEDBACK LOOPS	Slow – defects reported weeks or months after introduction	Sprint-level – daily standups, sprint review, retrospective	Real-time – automated alerts, production monitoring, instant pipeline feedback
AUTOMATION LEVEL	Low – mostly manual testing, limited automation	High – heavy automation of regression tests, acceptance tests within sprints	Very high – fully automated pipeline testing (unit, integration, security, performance) plus production validations
RESPONSIBILITY	Siloed – developers build, testers test	Shared – developers, testers, product owner all accountable for quality	Everyone – developers, operations, security engineers collectively own quality throughout the entire lifecycle

敏捷与DevOps生命周期中的质量

Quality in Agile & DevOps Lifecycles

Key Takeaways for Quality Leaders

- ❑ Shift-left to prevent defects, not just find them.
- ❑ Quality is a whole-team culture, not a role.
- ❑ CI/CD pipelines embed quality gates; automation is the engine.
- ❑ Production is part of the quality landscape—monitor, learn, improve.
- ❑ The lifecycle model fundamentally shapes your quality strategy.

优质领导者的关键点

- ❑ 向左移挡是为了防止缺陷，而不仅仅是发现缺陷。
- ❑ 质量是整个团队的文化，而不是一个角色。
- ❑ CI/CD流水线嵌入质量门；自动化是引擎。
- ❑ 生产是质量环境的一部分—监控、学习、改进。
- ❑ 生命周期模型从根本上塑造了你的质量战略。

今天的会议一览

Today's Session at a Glance

Brief note: This session is where theory meets reality.

Agenda:

- ❑ The Healthcare.gov story: what happened and why
- ❑ Root cause analysis through a quality lens
- ❑ Group discussion: what could have been different
- ❑ Week 1 assignment unveiled
- ❑ Wrap-up and look ahead to Week 2

简要说明: 本场是理论与现实的交汇点。

议程:

- ❑ Healthcare.gov 故事: 发生了什么以及原因
- ❑ 通过质量视角进行根本原因分析
- ❑ 小组讨论: 本可以有什么不同
- ❑ 第一周任务揭晓
- ❑ 总结并展望第二周

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

Healthcare.gov Launch (2013)

Key facts:

- ❑ October 1, 2013: launch of the U.S. federal health insurance marketplace
- ❑ Highly visible project; millions of users expected
- ❑ Estimated cost over \$300 million (front-end alone)
- ❑ Within hours of launch: severe performance problems, errors, inability to complete applications

Quote/headline screenshot: “Healthcare.gov Is Broken” (media reaction)

Healthcare.gov 启动 (2013年)

关键事实：

- ❑ 2013年10月1日：美国联邦健康保险市场启动
- ❑ 高度显眼的项目；预计有数百万用户
- ❑ 仅前端成本估计超过3亿美元
- ❑ 上线数小时内：严重的性能问题、错误、无法完成应用

引用/标题截图：“Healthcare.gov 坏了”（媒体反应）

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

The Scale of Failure OR “More Than Just Bugs”

Immediate consequences:

- ☐ Only a handful of users could enroll on day one
- ☐ Extensive error messages, timeouts, missing data
- ☐ Public trust severely damaged
- ☐ Emergency “tech surge” and rescue effort led by high-profile experts

Long-term: massive rework, congressional hearings, management shake-ups

失败的规模，或者“不仅仅是漏洞”

即时后果：

- ☐ 只有少数用户在第一天就能注册
- ☐ 大量错误信息、超时、数据缺失
- ☐ 公众信任严重受损
- ☐ 由知名专家领导的紧急“技术激增”与救援行动

长期来看：大规模重组、国会听证会、管理层大改

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

What Went Wrong – The Quality View

Key technical and process failures (simplified for analysis):

- ☐ Lack of end-to-end testing before launch
- ☐ No load/performance testing at scale
- ☐ Integration testing between multiple contractor components ignored
- ☐ Last-minute requirement changes without full regression
- ☐ No single technical leader / integrator responsible for overall quality

出了什么问题—质量观点

关键技术和流程失效（简化分析）：

- ☐ 发射前缺乏端到端测试
- ☐ 没有大规模的负载/性能测试
- ☐ 忽略多个承包商组件之间的集成测试
- ☐ 最后时刻的要求变更，但没有完全回归
- ☐ 没有单一的技术领导者或集成商负责整体质量

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

Mapping Failures to Quality Concepts

QA/QC/QM lens:

- ❑ QA (prevention): few enforced design reviews, chaotic requirements
- ❑ QC (detection): testing mostly in silos, minimal end-to-end/system testing
- ❑ QM (management): unclear quality ownership across multiple vendors

Cost of Quality: huge external failure costs (rework, reputation, legal/political)

Lifecycle lens: effectively a fragmented, late-integration waterfall without V-Model rigour

将失败映射到质量概念

QA/QC/QM透:

- ❑ QA (预防): 执行设计审查少, 需求混乱
- ❑ QC (检测): 主要在孤岛测试, 最小限度的端到端/系统测试
- ❑ 质量管理 (QM): 多家供应商之间的质量所有权不明确

质量代价: 巨大的外部失败成本 (重做、声誉、法律/政治)

生命周期视角: 实际上是一个零散、后期集成的瀑布式, 缺乏V模型的严谨性

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

The CoQ Breakdown OR “Cost of Quality: Where the Money Went”

- ❑ Prevention: low (insufficient early design validation)
- ❑ Appraisal: moderate (siloe component tests)
- ❑ Internal Failure: high (late discovery of integration bugs)
- ❑ External Failure: catastrophic (public failure, emergency rescue)

under-investment in prevention/appraisal led to massive external failure costs.

CoQ 分析或“质量成本：钱流向何方”

- ❑ 预防：低（早期设计验证不足）
- ❑ 评估：中等（分区测试）
- ❑ 内部故障：高（集成漏洞发现较晚）
- ❑ 外部故障：灾难性（公共故障，紧急救援）

预防和评估投资不足导致了巨大的外部失败成本。

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

Which Quality Attributes Failed?

- ☐ Reliability: system crashed or froze under load
- ☐ Performance Efficiency: pages took minutes to load
- ☐ Functional Suitability: core functions (enrollment) didn't work
- ☐ Usability: confusing flow, unclear error messages
- ☐ Security: concerns about personal data exposure added to panic

Did any attribute hold up? (almost none).

哪些质量属性失败了？

- ☐ 可靠性：系统在负载下崩溃或冻结
- ☐ 性能效率：页面加载只需几分钟
- ☐ 功能适应性：核心职能（招生）无效
- ☐ 可用性：流程混乱，错误信息不清晰
- ☐ 安全：个人信息暴露的担忧加剧了恐慌

有哪种属性能经得起时间推吗？（几乎没有）。

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

Traditional Lifecycle Pitfalls in Practice

Despite timeline pressures, the project followed a “big bang” integration approach.

No iterative feedback; independent contractor work integrated only at the end.

Even the V-Model’s basic traceability (requirements $\leftrightarrow$ acceptance tests) was missing.

Missing Spiral principle: no iterative risk-driven prototyping for the high-risk integration.

传统生命周期陷阱在实践中

尽管时间紧迫，项目仍采用了“爆炸”式集成方法。

没有迭代反馈;独立承包商的工作仅在最后阶段整合。

甚至V模型的基本可追溯性（需求 $\leftrightarrow$ 验收测试）也缺失了。

缺少螺旋原则：高风险集成不进行迭代风险驱动的原型。

案例分析：质量失败与作业启动

Case Analysis: Quality Failures & Assignment Kick-off

What Agile/DevOps Might Have Changed

Hypothetical counterfactuals:

Early MVP with scaled load testing in each sprint (shift-left)

Continuous integration across contractor teams

Automated quality gates preventing deployment of broken components

Canary releases to a small group before nationwide launch

Real-time monitoring and fast rollback capability

Key point: quality engineering, not late-stage heroics.

敏捷/DevOps 可能发生了哪些变化

假设性反事实：

早期MVP，每个冲刺（向左移位）都带了可扩展负载测试

承包商团队之间的持续集成

自动化质量门防止损坏组件的部署

金丝雀在全国上线前向少数人发布

实时监控和快速回滚能力

关键点：质量工程，而非后期英雄主义。

第一周作业—案例分析

Week 1 Assignment – Case Study Analysis

Assignment description:

Write a 2–3 page report analyzing an alternate case of your own interest using the frameworks from this week.

Required sections:

- ☐ Quality failure identification (attributes, CoQ)
- ☐ QA/QC/QM analysis
- ☐ Lifecycle approach critique (what worked badly)
- ☐ Proposed improvement plan using Agile/DevOps quality practices

Due date (Before Next Week's Class).

作业评分标准一览

Assignment Rubric at a Glance

- ❑ Accurate identification of quality issues and CoQ costs (25%)
- ❑ Correct application of QA/QC/QM concepts (25%)
- ❑ Insightful lifecycle critique (20%)
- ❑ Quality and practicality of improvement proposals (20%)
- ❑ Clarity, structure, and professional writing (10%)

摘要一大局

Summary – The Big Picture

Recap of Week 1 journey:

- ❑ Quality attributes, QA/QC/QM, Cost of Quality
- ❑ Traditional lifecycles – testing at the end is risky
- ❑ Agile & DevOps – built-in quality, shift-left, continuous everything
- ❑ Healthcare.gov – what happens when quality is neglected

The central lesson: Quality must be managed and engineered proactively, or it will cost you in the most public way.

第一周旅程回顾:

- ❑ 质量属性、质量保证/质量控制/质量管理、质量成本
- ❑ 传统的生命周期—最后测试风险很大
- ❑ 敏捷与DevOps—内置质量，左移，持续不断
- ❑ Healthcare.gov—当质量被忽视时会发生什么

核心教训是：质量必须主动管理和设计，否则将以最公开的方式付出代价。

下周预告：质量规划与基于风险的测试策略

Next Week: Quality Planning & Risk-Based Test Strategy

- ❑ How to build a quality plan from scratch
- ❑ Risk-driven test planning and prioritization
- ❑ Creating a Test Work Breakdown Structure (Test WBS)

问答 Q&A

Q&A and Final Discussion

Thank you!...